

aio-agent v2.1 — Resumen Ejecutivo

Propósito

aio-agent es un agente ligero de SRE que utiliza **function calling nativo** sobre el modelo local **Qwen2.5-7B-Instruct** (served by llama.cpp). Responde en español, ejecuta comandos Linux, lee archivos de configuración y logs, y escribe archivos bajo ciertas rutas permitidas.

Configuración inicial (Setup Wizard)

La primera ejecución ejecuta `setup.py`, un instalador en modo texto que permite elegir:

1. **LOCAL** — Usa Qwen2.5-7B-Instruct incluido en la ISO. Mínimo 8 GB RAM.
2. **CLOUD** — Seleccionar proveedor (DeepSeek, OpenAI, Anthropic, Google Gemini, Moonshot/Kimi, OpenRouter) e introducir API key.
3. **HÍBRIDO** — Consultas simples → local; consultas complejas → cloud.

El wizard también ajusta automáticamente los cores de CPU para llama.cpp, reservando ~20% para el sistema (por ejemplo, 14 threads en un VPS de 16 cores).

Modelo definitivo

- **Qwen2.5-7B-Instruct-Q4_K_M** en `http://localhost:8083/v1/chat/completions`, con ventana de contexto de 8K y `MAX_HISTORY_TOKENS=6000`.
- Servidor llama.cpp con `--jinja` y `-t 14` sobre 16 cores.
- Qwen2.5-7B-Instruct emite `tool_calls` válidos de forma consistente y completa planes multi-paso.

Cuantización del modelo

Cuantización	Velocidad (tok/s)	RAM/VRAM	Calidad observada	Caso de uso
Q4_K_M	57 prompt / 20 gen	4.7 GB	Referencia	Servidor productivo con suficiente memoria

La instancia desplegada actualmente usa **Qwen2.5-7B-Instruct-Q4_K_M**. Para evitar desbordamientos silenciosos de contexto, `agent.py` cuenta tokens con el endpoint real `/v1/tokenize` antes de comprimir o truncar el historial, en lugar de estimar con una relación fija de caracteres por token.

Modelos evaluados y descartados

Se probaron varios modelos <8B buscando una opción más ligera. Ninguno alcanzó la fiabilidad de function calling de Qwen2.5-7B-Instruct.

Modelo	Tamaño	tok/s	Function calling	Veredicto
Qwen3-8B Q3_K_M	3.9 GB	14.9	Funciona	Descartado — más lento que Qwen2.5-7B Q4_K_M

Modelo	Tamaño	tok/s	Function calling	Veredicto
Qwen3-4B (+jinja)	2.4 GB	42	Inconsistente	Descartado — responde de memoria, no usa tools
Qwen2.5-Coder-3B	1.8 GB	~33	Falla	Descartado — responde de memoria, no usa tools
Phi-4-Mini 3.8B	2.4 GB	46	Falla	Descartado — se niega a ejecutar comandos
Llama-3.2-3B	1.9 GB	37	Falla	Descartado — alucina tool calls con paths falsos
Gemma-3-4B-IT	2.4 GB	16	Falla	Descartado — alucina tool calls

Conclusión: **modelos <7B no son fiables para function calling en sysadmin.** Qwen2.5-7B-Instruct es el sweet spot para CPU.

Funcionalidades v2.1

Funcionalidad	Estado
Function calling nativo vía llama.cpp	[OK]
11 tools: run_command, read_file, write_file, web_search, git_operation, mcp_call, run_playbook, process_start, process_send, process_close, process_list	[OK]
Bucle conversacional con hasta 10 turnos tool→LLM	[OK]
Planificación y ejecución multi-paso sin intervención	[OK]
Memoria procedural (Skill-Pro, ICML 2026)	[OK]
Compresión real de contexto vía /v1/tokenize	[OK]
Recuperación de errores (apt/apt-get, locks)	[OK]
Sesión persistente entre reinicios	[OK]
Historial readline (flechas arriba/abajo, cursor)	[OK]
Ctrl+C interrumpe el turno actual sin salir	[OK]
Setup wizard: local/cloud/hybrid	[OK]
Seguridad OWASP: allowlist, confirmación destructiva, validación de input, audit log	[OK]
README.md y PDF ejecutivo actualizados	[OK]

Arquitectura

chat.py --> agent.py --> tools.py --> Qwen2.5-7B-Instruct vía llama.cpp:8083

- setup.py: wizard de configuración inicial.
- chat.py: bucle interactivo con readline y slash commands.
- agent.py: orquestador de function calling y memoria procedural.
- tools.py: definición y manejadores de herramientas.
- memory.py: cache procedural Skill-Pro.
- playbook.py: runner de playbooks YAML.
- process.py: gestión de procesos interactivos con PTY.

Uso

Configurar la primera vez:

```
python3 setup.py
```

Chat interactivo:

```
python3 chat.py
```

Ejemplos:

```
> muestra el uso de disco
> lee /var/log/syslog
> escribe un script de backup en /tmp/backup.sh
> reinicia nginx
```

Escribe salir, exit o quit para terminar. Ctrl+C durante un turno lo interrumpe y vuelve al prompt.

Seguridad

- Tool allowlist: rechaza `rm -rf /`, `dd` a dispositivos de bloque, `mkfs.*`, `fdisk`, `chmod 000`, etc.
- Human-in-the-loop: comandos destructivos (`rm`, `sudo rm`, `> /dev/sd*`, `format`, `dd`) piden confirmación (N por defecto, timeout 10 s).
- Validación de input: sanitiza caracteres de control, límite 1000 caracteres, detecta inyección de system prompt; los rechazos se registran en `audit.jsonl`.
- Protección de rutas: `write_file` bloquea `/etc`, `/boot`, `/sys`, `/proc`, `/dev`.
- Log de auditoría: cada invocación de tool y rechazo queda en `audit.jsonl`.
- Análisis completo en `SECURITY.md`.

Historial del proyecto

Semana 1 — RAG + híbrido

- Se empezó con **Qwen3-0.6B** + ChromaDB (e5-large embeddings) + corpus procedural de **955 documentos**.
- Batería de pruebas: solo **7/43 PASS (16%)**.
- Se intentó fine-tune del 0.6B con dataset SRE (500 ejemplos) vía Unsloth en GPU Lambda.
- El fine-tune falló por incompatibilidades de versiones (`trl`, `transformers`).
- Se abandonó el enfoque RAG por frágil y caro.

Semana 2 — Reset: function calling puro

- Se borraron los proyectos híbridos (~10 GB) y se empezó de cero.
- Se instaló **llama.cpp** compilado para CPU con `-j16`.
- Se descargó **Qwen3-8B** (bartowski instruct GGUF).
- Se construyó un agente en 3 archivos (~200 líneas) con function calling nativo.
- Tools iniciales: `run_command`, `read_file`, `write_file`.
- El agente funcionó en español, con **17 tok/s**.

Semana 3 — Features por capas

- Memoria procedural (Skill-Pro, ICML 2026): cache JSON para respuestas repetitivas.
- Planificación multi-paso: prompt con EJECUTA sin explicar.
- Compresión de contexto: contar tokens reales vía `/v1/tokenize`.

- Recuperación de errores: reintentos con apt-get si apt falla y gestión de locks de apt.
- Sesión persistente (data/session.json) + historial readline (data/.chat_history).
- Log de auditoría (audit.jsonl).

Semana 3 — Tools avanzadas

- web_search vía Firecrawl local.
- git_operation (status, commit, push, diff, log).
- mcp_call (conectar APIs externas vía MCP).
- run_playbook (ejecutar YAML secuencialmente).
- process_start / send / close / list (procesos interactivos con PTY).

Week 3 — Seguridad (OWASP)

- Tool allowlist: rm -rf /, dd, mkfs, fdisk, chmod 000 bloqueados.
- Human-in-the-loop: comandos destructivos piden confirmación.
- Input validation: sanitizar input, límite 1000 chars, anti-inyección.
- Se creó SECURITY.md con análisis OWASP completo.

Week 4 — Evaluación de modelos y setup wizard

- Se probaron varios modelos <8B; todos fallaron en function calling fiable para sysadmin.
- **Qwen2.5-7B-Instruct Q4_K_M** elegido como modelo definitivo.
- Se añadió setup.py con modos local/cloud/hybrid, selección de proveedor, API key y asignación automática de CPU.
- Se añadió Ctrl+C para interrumpir el turno actual sin salir del chat.

Estado final

- Modelo: **Qwen2.5-7B-Instruct Q4_K_M** (bartowski)
- Servidor: llama.cpp en :8083, --jinja, -c 8192, -t 14
- Velocidad: **57/20 tok/s** prompt/gen
- 11 tools: run_command, read_file, write_file, web_search, git_operation, mcp_call, run_playbook, process_start, process_send, process_close, process_list
- Memoria: cache procedural Skill-Pro
- Modos: local / cloud / híbrido (configurados por setup.py)
- Seguridad: allowlist OWASP, confirmación destructiva, validación de input, audit log
- Repo: github.com/ccarrillomanzanares/aios-agent

Documento generado el 23 de julio de 2026.